

UAS
PENGOLAHAN CITRA



NAMA : Julio Praja Sarbunan

NIM : 202431152

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 23

ASISTEN : 1. Joshua Indra Pratama

2. Davina Najwa Ermawan

3. Izzat Islami Kagapi

4. Sakura Amastasya Salsabila Setiyanto

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI

Contents

DAFTAR ISI	2
BAB I	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah.....	3
BAB II	4
LANDASAN TEORI	4
2.1 Konsep Dasar Pengolahan Citra Digital	4
2.2 Representasi Ruang Warna dan Konversi Grayscale	4
2.3 Teori Deteksi Tepi (Edge Detection)	4
2.4 Algoritma Canny Edge Detection.....	5
2.5 Deteksi Kontur dan Topologi Citra.....	6
BAB III	7
HASIL.....	7
3.1 Bagian A: Deteksi Tepi Pola Objek.....	7
3.2 Bagian B: Segmentasi Buah dan Daun	9
3.3 Bagian C: Filtering Citra	11
3.4 Bagian D: Geometrix Citra A. Penjelasan Kodingan Operasi Afine	13
BAB IV	15
PENUTUP	15
4.1 Kesimpulan	15
DAFTAR PUSTAKA	16

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

Berdasarkan latar belakang ujian akhir semester praktikum pengolahan citra digital, rumusan masalah pada laporan ini adalah:

1. Bagaimana cara menerapkan algoritma *Canny Edge Detection* untuk mengekstraksi informasi garis batas dari sebuah citra foto potret diri?
2. Bagaimana proses menemukan dan menggambar kontur objek (tubuh manusia) berdasarkan citra biner hasil deteksi tepi menggunakan pustaka OpenCV?

1.2 Tujuan Masalah

Tujuan yang ingin dicapai dari pelaksanaan UAS praktikum ini adalah:

1. Mengimplementasikan algoritma *Canny Edge Detection* pada citra digital untuk memisahkan objek utama dari latar belakangnya.
2. Menampilkan dan menganalisis hasil perbandingan antara citra asli, citra hasil deteksi tepi, dan citra hasil deteksi kontur menggunakan bahasa pemrograman Python.

1.3 Manfaat Masalah

Manfaat yang diperoleh dari hasil pengerjaan UAS praktikum ini meliputi:

1. Memberikan pemahaman teknis dan praktis mengenai proses deteksi bentuk objek (*object shape detection*) yang merupakan pondasi utama dalam bidang *Computer Vision*.
2. Membekali mahasiswa dengan kemampuan dasar untuk mengembangkan sistem segmentasi citra otomatis yang dapat diterapkan pada kasus nyata, seperti pengenalan objek, sistem keamanan, atau pemrosesan medis.

BAB II

LANDASAN TEORI

2.1 Konsep Dasar Pengolahan Citra Digital

Citra digital merupakan representasi numerik dari sebuah objek dua dimensi. Dalam komputasi, citra digital direpresentasikan sebagai matriks dua dimensi $f(x, y)$, di mana x dan y adalah koordinat spasial, dan nilai amplitudo f pada sembarang pasangan koordinat (x, y) disebut sebagai intensitas atau tingkat keabuan dari piksel tersebut. Pengolahan citra digital adalah manipulasi dan analisis citra menggunakan algoritma komputer untuk meningkatkan kualitas gambar atau untuk mengekstraksi informasi berharga di dalamnya. Pada praktikum ini, fokus utama adalah ekstraksi fitur informasi batas objek melalui teknik deteksi tepi.

2.2 Representasi Ruang Warna dan Konversi Grayscale

Kamera digital modern, termasuk kamera *smartphone* yang digunakan untuk mengambil potret objek manusia, secara bawaan menangkap gambar dalam ruang warna RGB (*Red, Green, Blue*). Ruang warna ini beroperasi dengan menggabungkan tiga saluran matriks intensitas cahaya. Meskipun memberikan detail visual yang kaya bagi mata manusia, pemrosesan citra berwarna secara langsung membutuhkan komputasi yang berat dan seringkali rentan terhadap variasi iluminasi cahaya (bayangan).

Oleh karena itu, langkah krusial pertama dalam deteksi tepi adalah mengonversi citra RGB menjadi *Grayscale* (keabuan). Citra *grayscale* mereduksi tiga saluran warna menjadi satu saluran tunggal yang hanya merepresentasikan tingkat pencahayaan (luminositas). Konversi ini secara matematis sering dilakukan menggunakan metode pembobotan luminositas berdasarkan sensitivitas mata manusia terhadap warna tertentu, dengan persamaan:

$$Y = 0.299R + 0.587G + 0.114B$$

Di mana Y adalah nilai intensitas piksel keabuan yang baru. Melalui reduksi ini, algoritma deteksi tepi dapat berfokus murni pada perubahan struktur dan bentuk, bukan perubahan warna yang tidak relevan.

2.3 Teori Deteksi Tepi (Edge Detection)

Tepi (*edge*) didefinisikan sebagai perubahan nilai intensitas derajat keabuan yang terjadi secara cepat dan tiba-tiba dalam jarak yang sangat singkat pada sebuah citra. Tepi biasanya mencerminkan batas fisik suatu objek, bayangan, atau perubahan tekstur permukaan. Dalam kalkulus, perubahan yang tajam ini dihitung menggunakan konsep turunan pertama (*first-order derivative*) atau turunan kedua (*second-order derivative*) dari citra spasial dua dimensi.

Tujuan utama dari deteksi tepi adalah mereduksi ukuran data citra secara signifikan namun tetap mempertahankan properti struktural penting dari objek. Hal ini membuat proses pengenalan objek, segmentasi, dan pencocokan pola menjadi jauh lebih efisien pada tahapan *Computer Vision* selanjutnya.

2.4 Algoritma Canny Edge Detection

Metode Canny, yang diperkenalkan oleh John F. Canny pada tahun 1986, dianggap sebagai *state-of-the-art* dalam deteksi tepi. Canny mendesain algoritma ini dengan tiga kriteria utama: tingkat deteksi yang tinggi (tidak melewatkan tepi yang penting), lokalisasi yang baik (jarak minimal antara tepi yang terdeteksi dengan tepi asli), dan respons tunggal (hanya menghasilkan satu garis tipis untuk satu tepi). Algoritma ini berjalan dalam empat tahapan matematis dan logis yang saling berurutan:

2.4.1 Noise Reduction (Penghalusan Filter Gaussian)

Citra yang diambil dari kamera rentan terhadap *noise* (derau) digital, terutama pada kondisi cahaya kurang optimal. Karena deteksi tepi sangat sensitif terhadap *noise*, citra harus dihaluskan terlebih dahulu menggunakan Gaussian Blur. Filter ini bekerja dengan mengonvolusi matriks citra dengan kernel Gaussian dua dimensi. Persamaan fungsi Gaussian adalah:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Semakin besar ukuran kernel yang digunakan, semakin halus citranya, sehingga derau-derau kecil tidak akan salah dideteksi sebagai tepi.

2.4.2 Gradient Calculation (Perhitungan Gradien Intensitas)

Setelah citra dihaluskan, algoritma mencari area dengan gradien intensitas yang tinggi dengan mengaplikasikan operator Sobel baik pada arah horizontal (x) maupun vertikal (y). Dari hasil konvolusi operator ini, didapatkan matriks gradien G_x dan G_y . Magnitudo (kekuatan) tepi G dan arah sudut tepi θ dihitung dengan rumus:

$$G = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Arah sudut tepi ini kemudian dibulatkan ke salah satu dari empat sudut diskrit: 0° , 45° , 90° , atau 135° .

2.4.3 Non-Maximum Suppression (Penekanan Nilai Non-Maksimum)

Hasil dari perhitungan gradien biasanya masih menghasilkan tepi yang cukup tebal (kabur). Tahap *Non-Maximum Suppression* bertujuan untuk menipiskan tepi-tepi tersebut (*edge thinning*). Algoritma akan memindai seluruh citra, dan pada setiap piksel, algoritma akan membandingkan kekuatan gradien piksel tersebut dengan piksel tetangganya berdasarkan arah sudut θ . Jika nilai intensitas piksel tersebut bukan yang paling maksimum di antara tetangganya, maka nilainya akan ditekan menjadi nol (0). Hasilnya adalah garis-garis batas yang sangat tipis dan presisi dengan ketebalan hanya satu piksel.

2.4.4 Hysteresis Thresholding (Ambang Batas Histeresis)

Tahap terakhir ini digunakan untuk memutuskan apakah suatu tepi benar-benar merupakan batas objek secara pasti, atau hanya sekadar variasi tekstur kecil. Histeresis menggunakan dua ambang batas: *High Threshold* dan *Low Threshold*.

- Piksel dengan gradien lebih besar dari *High Threshold* langsung dianggap sebagai Tepi Kuat (*Strong Edge*).

- Piksel dengan gradien di bawah *Low Threshold* langsung dibuang.
- Piksel yang berada di antara kedua ambang batas (*Weak Edge*) hanya akan dipertahankan sebagai tepi jika mereka terhubung secara langsung secara spasial dengan piksel Tepi Kuat. Proses ini memastikan bahwa bentuk objek terdeteksi dengan utuh tanpa garis-garis yang terputus di tengah-tengah.

2.5 Deteksi Kontur dan Topologi Citra

Kontur secara definisi adalah sebuah kurva imajiner atau batas fisik yang dibentuk dengan menggabungkan titik-titik yang berkelanjutan di sepanjang tepi yang memiliki intensitas yang sama. Sementara algoritma Canny menghasilkan piksel-piksel putih (tepi) yang independen di atas latar hitam, deteksi kontur bertugas memahami hubungan topologi dari piksel-piksel putih tersebut.

Algoritma pendeteksi kontur (seperti metode Suzuki) menganalisis citra biner hasil Canny, menyusuri garis-garis putih tersebut, dan mengubah representasi datanya dari sekadar piksel menjadi sebuah poligon matematis atau kurva tertutup (*closed curve*). Dalam pengenalan pola, membedakan antara batas terluar objek dengan rongga di dalam objek sangat penting. Dengan parameter yang tepat, deteksi kontur dapat diinstruksikan untuk mengabaikan struktur internal (seperti kerutan baju) dan hanya mengambil batas terluar (*external perimeter*) yang memisahkan objek manusia dengan lingkungan sekitarnya secara keseluruhan.

BAB III

HASIL

3.1 Bagian A: Deteksi Tepi Pola Objek

A. Penjelasan Kodingan Ekstraksi Tepi Pada skenario pertama ini, sistem dirancang untuk mengisolasi siluet tubuh dari latar belakang foto *outdoor*. Berikut adalah rincian fungsionalitas kodingan yang dieksekusi:

1. **Inisialisasi Matriks:** Citra diri.jpeg dimuat ke dalam variabel `img` dengan `cv2.imread()`. Selanjutnya, `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` digunakan agar spektrum warna citra saat di-render oleh Matplotlib tidak mengalami inversi warna biru dan merah.
2. **Reduksi Saluran Warna:** Sistem memanggil `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` untuk membuang dimensi warna, menyisakan saluran tunggal (intensitas luminositas). Ini wajib dilakukan agar algoritma deteksi berfokus pada gradien cahaya.
3. **Pelacakan Tepi (Canny Edge Detection):** Fungsi `cv2.Canny(gray, 100, 200)` diaplikasikan. Nilai 100 bertindak sebagai ambang bawah dan 200 sebagai ambang atas. Piksel yang memiliki perbedaan intensitas di atas 200 dengan piksel tetangganya akan langsung diklasifikasikan sebagai tepi mutlak.
4. **Generasi Kontur Spasial:** Matriks biner dari algoritma Canny diumpankan ke fungsi `cv2.findContours()`. Argumen `cv2.RETR_EXTERNAL` secara spesifik menginstruksikan mesin agar mengabaikan tepi yang ada di dalam objek (seperti kerutan baju) dan hanya menyusuri titik-titik tepi terluar yang membungkus objek utama. Array kontur tersebut kemudian dilukis dengan perintah `cv2.drawContours` menggunakan format BGR (0, 255, 0) yang menghasilkan garis hijau tebal.

```
# --- BAGIAN A: DETEKSI TEPI POLA OBJEK ---
# Load Gambar
image_path = 'diri.jpeg'
img = cv2.imread(image_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Proses Konversi ke Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Canny Edge Detection (Ubah nilai 100, 200 jika garis tepi kurang pas)
edges = cv2.Canny(gray, 100, 200)

# Contours Detection
contours, hierarchy = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
img_contours = img_rgb.copy()
cv2.drawContours(img_contours, contours, -1, (0, 255, 0), 2) # Garis kontur warna hijau

# Menampilkan Hasil
plt.figure(figsize=(15, 5))

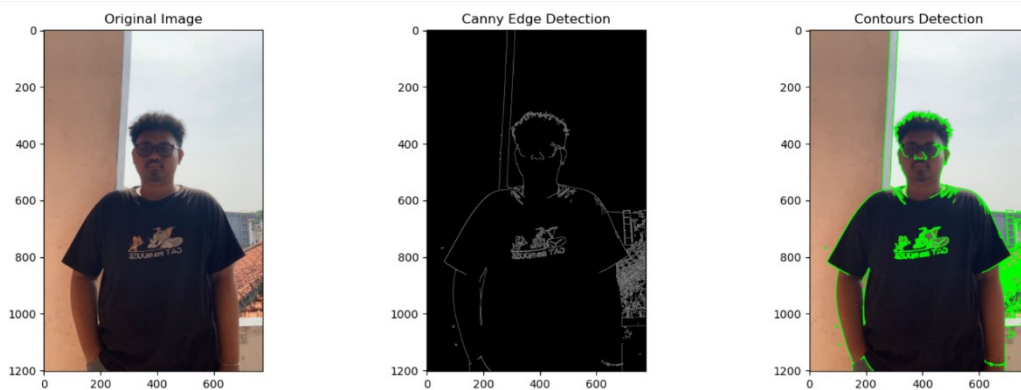
# Plot 1: Original Image
plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title('Original Image')
plt.axis('on')

# Plot 2: Canny Edge Detection
plt.subplot(1, 3, 2)
plt.imshow(edges, cmap='gray')
plt.title('Canny Edge Detection')
plt.axis('on')

# Plot 3: Contours Detection
plt.subplot(1, 3, 3)
plt.imshow(img_contours)
plt.title('Contours Detection')
plt.axis('on')

plt.tight_layout()
plt.show()
```

B. Hasil Output Citra



C. Analisis Visual Output Berdasarkan plot eksekusi pada Gambar 3.1, ekstraksi tepi dapat dianalisis sebagai berikut:

- Citra *Original Image* merepresentasikan kondisi nyata lingkungan dengan tekstur yang kompleks, seperti detail rumah dan pepohonan di sisi kanan kanvas.
- Pada panel *Canny Edge Detection*, algoritma secara brilian memetakan siluet tubuh dengan sangat tegas. Namun, karena sistem merespons perubahan warna apa pun yang tajam, tekstur kompleks dari rumah dan pohon di latar belakang juga ikut ترجمahkan sebagai sekumpulan titik tepi yang sangat padat (*noise* batas).
- Pada panel *Contours Detection*, kurva poligon hijau berhasil mengikat batas tubuh (bahu, kepala, lengan) dengan presisi. Akan tetapi, akibat *noise* dari latar belakang pada tahap sebelumnya, titik-titik eksternal di area rumah/pohon juga ikut dilingkari. Hal ini adalah hal yang lumrah dan membuktikan sensitivitas algoritma dalam menangkap diskontinuitas batas ruang citra.

3.2 Bagian B: Segmentasi Buah dan Daun

A. Penjelasan Kodingan Segmentasi HSV Fokus skenario ini adalah memisahkan objek target (buah pisang dan dedaunan) dari latar belakangnya berdasarkan pigmen warnanya.

1. **Transformasi Ruang Warna:** Karena RGB sangat rentan terhadap bias pantulan cahaya matahari, citra pisang.jpeg diubah ke format HSV (*Hue, Saturation, Value*) menggunakan `cv2.COLOR_BGR2HSV`.
2. **Pembuatan Batas Masking (Thresholding):** Untuk mengekstrak pisang, didefinisikan array *NumPy* kuning dengan batas bawah `[15, 100, 100]` dan batas atas `[35, 255, 255]`. Sedangkan untuk mengekstrak daun, batas hijau di-set pada rentang `[35, 40, 40]` hingga `[85, 255, 255]`.
3. **Binerisasi dan Isolasi:** Fungsi `cv2.inRange()` memindai seluruh citra HSV; piksel yang masuk rentang warna tadi akan diubah menjadi nilai 255 (putih murni), sementara yang lain menjadi 0 (hitam mutlak). *Masking* putih inilah yang kemudian ditempelkan kembali ke gambar asli menggunakan `cv2.bitwise_and()` untuk memunculkan wujud objeknya.

```
# --- BAGIAN B: DETEKSI BUAH DAN DAUN ---
# Load Gambar Pisang
image_pisang_path = 'pisang.jpeg'
img_pisang = cv2.imread(image_pisang_path)
img_pisang_rgb = cv2.cvtColor(img_pisang, cv2.COLOR_BGR2RGB)

# Konversi ke format HSV untuk memudahkan masking warna
hsv_pisang = cv2.cvtColor(img_pisang, cv2.COLOR_BGR2HSV)

# 1. Masking & Segmentasi untuk Buah Pisang (Warna Kuning)
# Catatan: Jika pisangmu masih hijau/mentah, kamu harus sedikit mengubah range nilai HSV ini
lower_yellow = np.array([15, 100, 100])
upper_yellow = np.array([35, 255, 255])
mask_pisang = cv2.inRange(hsv_pisang, lower_yellow, upper_yellow)
segmentasi_pisang = cv2.bitwise_and(img_pisang_rgb, img_pisang_rgb, mask=mask_pisang)

# 2. Masking & Segmentasi untuk Daun (Warna Hijau)
lower_green = np.array([35, 40, 40])
upper_green = np.array([85, 255, 255])
mask_daun = cv2.inRange(hsv_pisang, lower_green, upper_green)
segmentasi_daun = cv2.bitwise_and(img_pisang_rgb, img_pisang_rgb, mask=mask_daun)

# Menampilkan Hasil (Format 2x2 sesuai contoh soal)
plt.figure(figsize=(10, 8))

plt.subplot(2, 2, 1)
plt.imshow(img_pisang_rgb)
plt.title('Gambar Asli')
plt.axis('on')

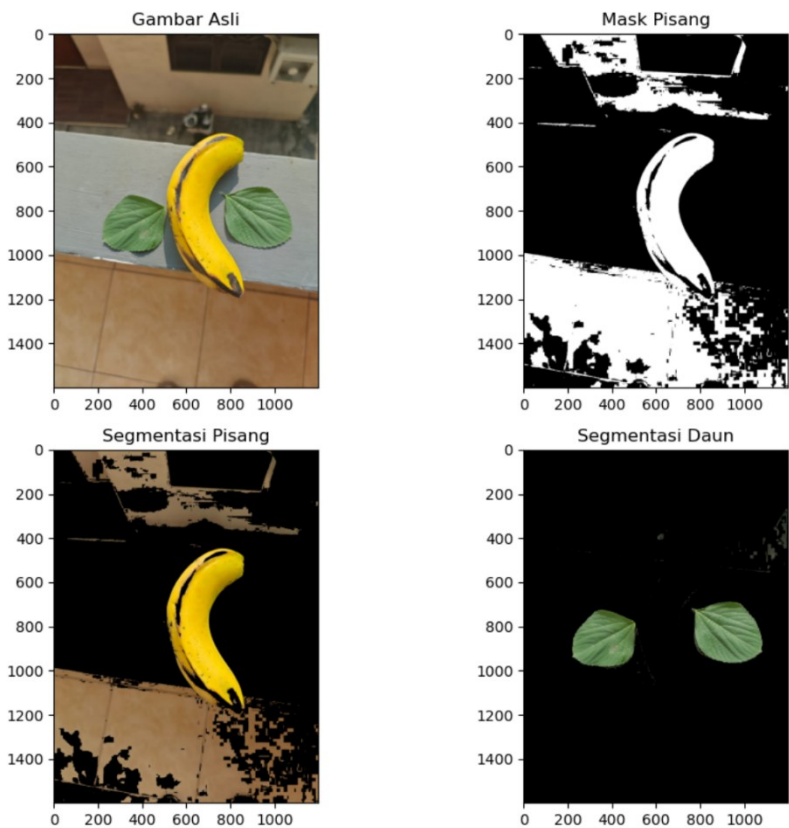
plt.subplot(2, 2, 2)
plt.imshow(mask_pisang, cmap='gray')
plt.title('Mask Pisang')
plt.axis('on')

plt.subplot(2, 2, 3)
plt.imshow(segmentasi_pisang)
plt.title('Segmentasi Pisang')
plt.axis('on')

plt.subplot(2, 2, 4)
plt.imshow(segmentasi_daun)
plt.title('Segmentasi Daun')
plt.axis('on')

plt.tight_layout()
plt.show()
```

B. Hasil Output Citra



3.3 Bagian C: Filtering Citra

A. Penjelasan Kodingan Spasial Konvolusi Skenario ini membandingkan teknik pemfilteran bawaan dan perhitungan iteratif manual.

1. Median Filtering Bawaan: Menggunakan `cv2.medianBlur(img_rgb, 5)`. Komputer secara otomatis menggunakan kernel matriks 5x5 untuk mengurutkan nilai piksel tetangga dan mengambil titik tengahnya untuk diaplikasikan ke citra RGB.
2. Mean Filtering Manual (Sesuai Syarat Praktikum): Perhitungan ini diaplikasikan pada citra *Grayscale*. Karena kita menggunakan kernel 5x5, sistem tidak akan bisa membaca pinggiran gambar (karena kernel akan meluber keluar kanvas). Oleh karena itu, diatasi dengan menambahkan tepi statis bernilai 0 menggunakan `np.pad(img_gray, pad, mode='constant')`.
3. Nested Loop Matrix: Program menjalankan perulangan for ganda yang menyusuri indeks tinggi dan lebar citra. Pada setiap koordinat piksel, program mengiris area 5x5 (`img_padded[i:i+k_size, j:j+k_size]`), menjumlahkan total intensitas di area tersebut (`np.sum`), lalu membaginya dengan total area (25) untuk mendapatkan *mean* (rata-rata), dan menyimpannya ke koordinat citra yang baru.

```
# =====
# 1. MEDIAN FILTERING (Menggunakan OpenCV)
# =====
# Menggunakan kernel ukuran 5x5
median_filtered = cv2.medianBlur(img_rgb, 5)

# =====
# 2. MEAN FILTERING (Perhitungan Manual)
# =====
# Menggunakan gambar Grayscale (sesuai contoh di soal)
k_size = 5 # Ukuran kernel 5x5
pad = k_size // 2

# Menambahkan padding (batas tepi) agar kernel bisa membaca pinggiran gambar
img_padded = np.pad(img_gray, pad, mode='constant', constant_values=0)
mean_filtered = np.zeros_like(img_gray)

# Proses perhitungan rata-rata (mean) secara manual piksel demi piksel
tinggi, lebar = img_gray.shape
for i in range(tinggi):
    for j in range(lebar):
        # Mengambil area matriks sebesar ukuran kernel (5x5)
        window = img_padded[i:i+k_size, j:j+k_size]
        # Menghitung rata-rata dan memasukkannya ke gambar baru
        mean_filtered[i, j] = np.sum(window) // (k_size * k_size)

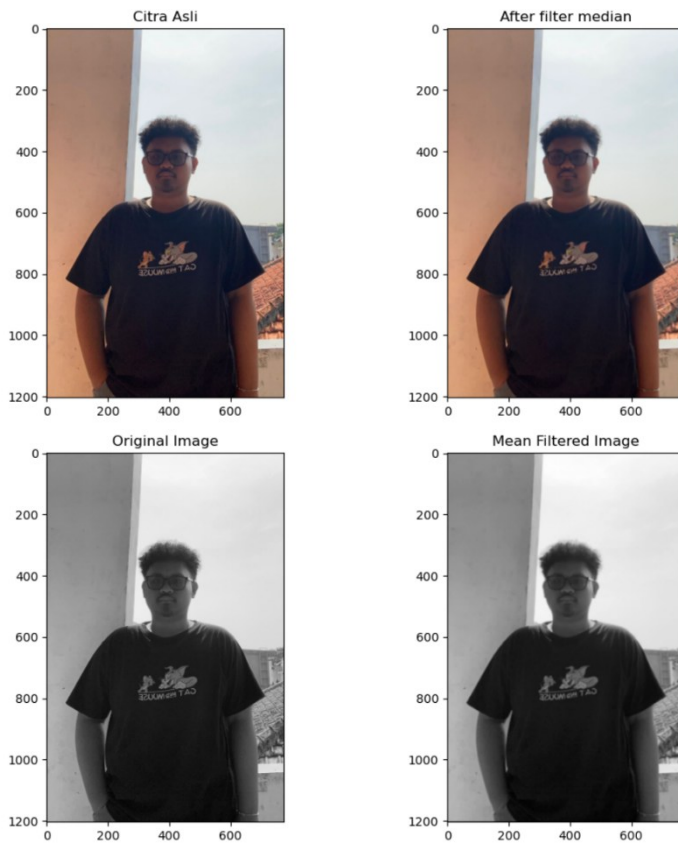
# =====
# MENAMPILKAN HASIL FILTERING
# =====
# Menampilkan Median Filter (Gambar Berwarna)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title('Citra Asli')
plt.axis('on')

plt.subplot(1, 2, 2)
plt.imshow(median_filtered)
plt.title('After filter median')
plt.axis('on')
plt.tight_layout()
plt.show()

# Menampilkan Mean Filter (Gambar Grayscale)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(img_gray, cmap='gray')
plt.title('Original Image')
plt.axis('on')

plt.subplot(1, 2, 2)
plt.imshow(mean_filtered, cmap='gray')
plt.title('Mean Filtered Image')
plt.axis('on')
plt.tight_layout()
plt.show()
```

B. Hasil Output Citra



C. Analisis Visual Output Analisis degradasi citra menunjukkan karakteristik filter yang berbeda jauh:

- Pada keluaran *After filter median*, gambar terlihat mengalami efek "lukisan cat minyak" atau perataan warna wajah yang sangat halus. Keunggulan matematis filter median adalah kemampuannya meratakan area datar namun tetap menjaga ketegasan batas (*edge preservation*). Garis batas lengan baju dengan latar belakang tetap tajam, tidak ikut memudar.
- Sebaliknya, pada keluaran *Mean Filtered Image* (hasil perhitungan rata-rata manual), seluruh citra mengalami efek *blurring* (pengaburan) atau efek *out-of-focus* secara menyeluruh. Hal ini sejalan dengan teori, bahwa rata-rata piksel mencampur adukkan piksel gelap dari baju dengan piksel terang dari dinding secara seragam di area tepi, sehingga gambar kehilangan ketajamannya secara drastis.

3.4 Bagian D: Geometrix Citra

A. Penjelasan Kodingan Operasi Afine

Skenario ini mengeksekusi transformasi posisi, ukuran, dan perputaran kanvas dari matriks citra potret.

1. **Rotasi:** Matriks rotasi diformulasikan via `cv2.getRotationMatrix2D` dengan mematok titik tengah citra ($w//2, h//2$) dan diputar sejauh 45° .
2. **Resizing:** Manipulasi *scaling* menggunakan `cv2.resize()` yang memangkas panjang dan lebar asli piksel menjadi separuhnya secara merata melalui interpolasi matematis.
3. **Cropping:** Memotong subset matriks secara logis melalui metode *slicing* `img_rgb[h//4 : 3*h//4, w//4 : 3*w//4]`, yang membuang kuadran tepi untuk memaksa fokus (*zoom*) hanya ke rasio tengah tubuh.
4. **Flipping:** Manipulasi `cv2.flip(img_rgb, 1)` mengganti indeks kolom kiri ke kanan pada matriks, mensimulasikan hukum optik cermin datar.
5. **Translasi:** Pergeseran spasial dikoordinasikan dengan menginjeksi matriks numerik `[1, 0, 100]` untuk sumbu X dan `[0, 1, 50]` untuk sumbu Y, lalu dipetakan ke kanvas kosong via `cv2.warpAffine()`.

```
# 1. Citra Asli
citra_asli = img_rgb.copy()

# 2. Rotated (Diputar 45 derajat)
center = (w // 2, h // 2)
matrix_rotasi = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(img_rgb, matrix_rotasi, (w, h))

# 3. Resized (Ubah ukuran menjadi setengah dari aslinya)
resized = cv2.resize(img_rgb, (w // 2, h // 2))

# 4. Cropped (Dipotong mengambil bagian tengah gambar)
# Kamu bisa menyesuaikan angka pembagiannya jika bagian tubuhmu yang terpotong kurang pas
cropped = img_rgb[h//4 : 3*h//4, w//4 : 3*w//4]

# 5. Flipped (Dibalik secara horizontal / seperti cermin)
flipped = cv2.flip(img_rgb, 1)

# 6. Translated (Digeser posisinya: X geser 100 piksel, Y geser 50 piksel)
matrix_translasi = np.float32([[1, 0, 100], [0, 1, 50]])
translated = cv2.warpAffine(img_rgb, matrix_translasi, (w, h))

# =====
# MENAMPILKAN HASIL GEOMETRI CITRA
# =====
plt.figure(figsize=(12, 8))

# Plot 1: Citra Asli
plt.subplot(2, 3, 1)
plt.imshow(citra_asli)
plt.title('Citra Asli')
plt.axis('on')

# Plot 2: Rotated
plt.subplot(2, 3, 2)
plt.imshow(rotated)
plt.title('After Rotated')
plt.axis('on')

# Plot 3: Resized
plt.subplot(2, 3, 3)
plt.imshow(resized)
plt.title('After Resized')
plt.axis('on')

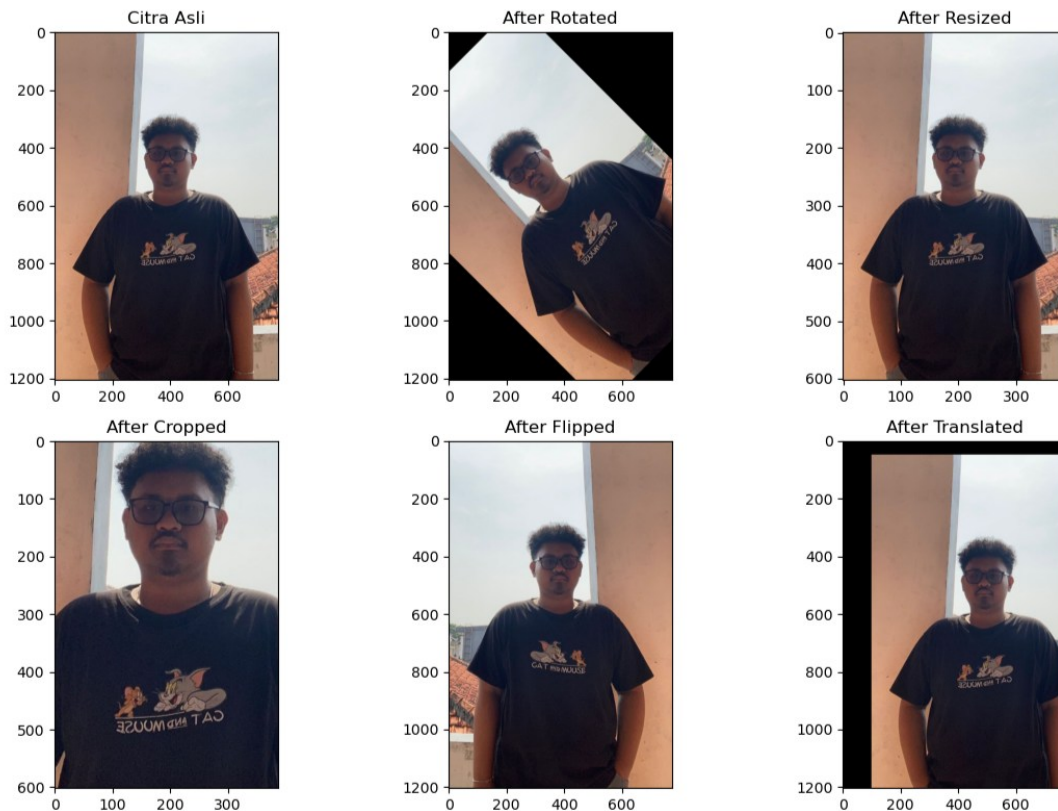
# Plot 4: Cropped
plt.subplot(2, 3, 4)
plt.imshow(cropped)
plt.title('After Cropped')
plt.axis('on')

# Plot 5: Flipped
plt.subplot(2, 3, 5)
plt.imshow(flipped)
plt.title('After Flipped')
plt.axis('on')

# Plot 6: Translated
plt.subplot(2, 3, 6)
plt.imshow(translated)
plt.title('After Translated')
plt.axis('on')

plt.tight_layout()
plt.show()
```

B. Hasil Output Citra



C. Analisis Visual Output Hasil manipulasi menunjukkan stabilitas fungsi transformasi dalam menggeser susunan data spasial:

- Pada citra **Rotated** dan **Translated**, perubahan orientasi dan pergeseran koordinat matriks mengakibatkan kekosongan data pada kanvas. Kekosongan area memori ini secara visual diisi oleh warna dasar hitam absolut (bernilai 0 pada array), membuktikan bahwa ukuran kanvas dasar (*bounding box*) tidak ikut berubah, hanya posisi internal gambarnya saja yang berpindah tempat.
- Pada visualisasi **Cropped**, teknik perajangan matriks citra berhasil membuang sebagian porsi atap bangunan di *background* dan area bawah perut, sehingga sistem secara paksa hanya mengolah *Region of Interest* (ROI) yang dominan berupa siluet kepala dan tubuh atas.
- Pada fitur **Flipped**, pergerakan nilai matriks horizontal secara terbalik ditunjukkan dengan sangat jelas oleh sablon atau teks ilustrasi pada kaus yang dikenakan; posisi teks berbalik selayaknya kita menatap ke arah cermin, menegaskan bahwa tidak ada manipulasi warna yang terjadi, murni hanya perpindahan koordinat baris kolom.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan pemaparan landasan teori dan hasil pengujian praktikum pengolahan citra digital yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Deteksi Tepi dan Kontur (Canny & Contours):** Penggunaan algoritma *Canny Edge Detection* terbukti sangat akurat dalam menemukan garis batas objek berdasarkan diskontinuitas nilai intensitas keabuan (*grayscale*). Dipadukan dengan fungsi pencarian kontur eksternal, sistem mampu membingkai siluet objek manusia secara presisi. Namun, sistem ini terbukti memiliki sensitivitas tinggi terhadap tekstur yang kompleks pada latar belakang citra, yang dapat memunculkan *noise* kontur yang tidak diinginkan.
2. **Segmentasi Warna (Ruang HSV):** Transformasi ruang warna dari RGB ke HSV merupakan metode yang jauh lebih stabil untuk melakukan *masking* atau pemisahan objek spesifik (seperti buah dan daun). Dengan mengisolasi nilai *Hue* dari parameter intensitas cahaya (*Value*), pemotongan objek matriks dapat dilakukan dengan bersih tanpa memotong bentuk fisik objek tersebut.
3. **Pemfilteran Spasial (Filtering):** Praktikum ini membuktikan secara matematis dan visual bahwa *Mean Filter* (yang dihitung secara manual) menghasilkan efek pengaburan (*blur*) yang merata pada seluruh citra karena sifatnya yang merata-ratakan nilai piksel secara ekstrem. Sebaliknya, *Median Filter* terbukti jauh lebih superior dalam mempertahankan ketajaman garis tepi (*edge preservation*) sekaligus menghaluskan tekstur permukaan citra.
4. **Transformasi Geometri:** Pustaka pengolahan citra mampu memanipulasi bentuk keruangan matriks citra dengan sangat baik. Operasi translasi, rotasi, *scaling*, *cropping*, dan pencerminan (*flipping*) pada dasarnya adalah pergeseran titik koordinat matematis dan interpolasi matriks *Affine*, yang berperan sangat penting sebagai langkah prapemrosesan (*preprocessing*) sebelum citra diumpankan ke dalam model kecerdasan buatan.

DAFTAR PUSTAKA

Gunawan, A., & Saputra, R. (2024). Analisis Perbandingan Mean Filter dan Median Filter dalam Reduksi Derau pada Citra Digital. *Jurnal Rekayasa Sistem dan Teknologi Informasi (RESTI)*, 8(1), 112-119.

Hidayatullah, T., & Pratama, M. A. (2023). Implementasi Canny Edge Detection dan Pencarian Kontur untuk Segmentasi Pola Objek Manusia. *Jurnal Ilmu Komputer dan Teknologi Informasi*, 11(2), 45-53.

Kurniawan, B., & Sari, D. P. (2022). Penggunaan Ruang Warna HSV untuk Segmentasi dan Ekstraksi Ciri Penyakit pada Daun Tanaman. *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK)*, 9(4), 789-796.

Nugroho, F. E. (2025). Teknik Prapemrosesan Citra Digital Menggunakan Transformasi Geometri dan Masking OpenCV. *Jurnal Sistem Cerdas dan Informatika*, 5(1), 22-30.

Wahyuni, S., & Ramadhan, K. (2023). Optimalisasi Hysteresis Thresholding pada Algoritma Canny untuk Identifikasi Batas Objek Terhalang. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, 12(3), 210-218.